

PD-TDM: SLO-AWARE PHASE ALLOCATION FOR SHARED-GPU LLM SERVING

[Author list pending]

[Affiliations pending — internal draft]

ABSTRACT

Prefill and Decode share an accelerator but consume its service capacity in different ways: Prefill determines time to first token (TTFT), while Decode determines time per output token (TPOT). Chunked prefill bounds the work in one iteration, yet a mixed batch still leaves the cross-iteration P/D service share implicit. We present PD-TDM, an explicit phase-level allocator for shared-GPU LLM serving. It keeps weights and KV state local, bounds Prefill continuation, and selects Prefill-only or Decode-only service quanta according to a configurable phase tendency and runtime safety constraints. On Qwen3-8B with two Ascend 910B3 NPUs, the evaluated allocator improves strict joint-SLO attainment over same-platform vLLM-Ascend chunked prefill by 34.8 percentage points on a conversation burst and 73.9 points on a code burst. A reconstructed MaaS trace shows lower TTFT, while Decode-heavy tests expose the expected boundary. A two-timescale SLO-pressure update for the phase tendency is formulated as an extension and is evaluated separately from these configured operating points.

Index Terms— Large language model serving, prefill–decode scheduling, temporal multiplexing, service-level objectives, Ascend NPU

1. INTRODUCTION

Online large-language-model (LLM) serving must control both time to first token (TTFT) and time per output token (TPOT). Prefill processes the prompt and is typically compute intensive; decode repeatedly reads a growing KV cache and is often memory intensive. A scheduler therefore needs to advance queued prompts without creating long gaps for active generations.

Continuous batching rebuilds the batch at every iteration [1, 2]. Chunked prefill further limits prompt work and mixes a bounded prefill chunk with decode tokens, reducing generation stalls [3, 4]. This bound controls how much prefill work enters one iteration, but does not independently specify how often Prefill is served across iterations: under a decode-first token budget, available Prefill capacity depends on the current Decode batch. Thus, mixed-batch composition and cross-iteration phase allocation are related but distinct control variables.

Physical prefill–decode disaggregation can provision the phases independently, but requires distinct instances and KV

transfer [5, 6]. Spatial multiplexing instead depends on concurrent kernels and controllable compute partitioning. TaiChi shows that SLO-aware Goodput can also be improved by combining aggregation and disaggregation with heterogeneous P-heavy/D-heavy instances and request-level latency shifting [7]. We study a complementary resource regime: both phases must use the same GPU group, without provisioning independent P/D instances or relying on spatial partitioning, while weights and KV state remain local.

We present PD-TDM, a phase-level service allocator for this shared-GPU regime. It separates state sharing from execution: chunked continuation and local KV state are retained, but an iteration is assigned to either Prefill or Decode before its phase-specific batch is built. A token-bucket selector turns a target Prefill tendency into bounded service quanta, while queue fallbacks, phase-length constraints, and KV-pressure guards preserve work conservation and safety. The design also admits a slower SLO-pressure loop that updates the target tendency; this extension is kept separate from the configured operating points used for the main evaluation.

Our contributions are: (1) formulating cross-iteration P/D service allocation as an explicit control dimension separate from the per-iteration chunk bound; (2) implementing bounded phase-pure execution and a configurable SLO-pressure allocator on a shared TP model instance; and (3) characterizing the Prefill-pressure benefit, SLO ceiling, and Decode-heavy boundary on two Ascend 910B3 NPUs. The reported three-seed results establish the end-to-end effect of configured phase allocation; the adaptive update is treated as a separately identified extension rather than being credited for these results.

2. BACKGROUND AND RELATED WORK

Aggregated serving. Orca introduced iteration-level scheduling, and vLLM’s PagedAttention enables memory-efficient continuous batching [1, 2]. Long prefills can nevertheless delay generation. Sarathi-Serve bounds prefill chunks and constructs decode-maximal mixed batches; Dynamic Split-Fuse similarly splits and combines prompt and generation work [3, 4]. PD-TDM retains bounded Prefill as a foundation, but treats the opportunity assigned to each phase across iterations as a separate control variable. Our measured baseline is the native chunked-prefill path in vLLM-Ascend on the same NPUs, denoted vLLM-Ascend CP; we do not claim a

cross-platform comparison to the Sarathi-Serve system.

SLO-aware P/D allocation. DistServe makes the TTFT/TPOT and Goodput tradeoff explicit for disaggregated serving [6]. TaiChi is the closest recent work in optimization objective: it adapts a mixture of P-heavy and D-heavy instances and uses request-level placement and latency shifting to cover different SLO regimes [7]. PD-TDM does not claim SLO-aware Goodput or latency shifting as new objectives. Its distinction is the control regime and granularity: phase-level service selection over one fixed shared-GPU execution path, rather than request placement or migration across heterogeneous instances. The proposed adaptive target-ratio update is therefore a scheduler mechanism to validate, not a claim that PD-TDM replaces TaiChi’s deployment-level optimization.

Disaggregation and spatial multiplexing. Splitwise and DistServe place prefill and decode on different instances, while Mooncake builds a KV-cache-centric distributed architecture [5, 6, 8]. These designs gain independent phase capacity at the cost of additional replicas and state movement. MuxWise and semi-PD share storage while concurrently partitioning GPU compute between the phases [9, 10]. Their spatial isolation is orthogonal to PD-TDM, which gives the entire TP replica to one phase at a time.

Temporal orchestration. EcoServe is the closest prior design: it temporally disaggregates phases within an instance and rolls activation across multiple cooperating instances to keep prefill available [11]. PD-TDM does not claim to originate temporal disaggregation. It instead studies bounded, iteration-granular phase allocation when both phases remain on one shared GPU group, and explicitly reports the Decode-pressure region in which this choice fails.

3. PROBLEM AND METRIC

Let an iteration have token budget B , prefill bound C , and D_t active decode tokens. A decode-first mixed scheduler can admit prefill work $P_t \leq \min(C, B - D_t)$. Thus C bounds a chunk, but the current decode batch implicitly determines the prefill service left in that iteration. PD-TDM separates the controls: a prefill iteration admits at most $P_t \leq \min(C, B)$ prompt tokens, while a phase share determines how frequently Prefill is selected when both queues contain work. In other words, the mixed policy chooses (P_t, D_t) inside an iteration, whereas PD-TDM first chooses a phase $u_t \in \{P, D\}$ and then constructs the corresponding batch.

The evaluated implementation exposes the phase share as a target r and uses a token bucket to smooth it over iterations. An adaptive extension can update r on a slower window from the difference between Prefill and Decode SLO pressure. One simple form is

$$r_{k+1} = \Pi_{[r_{\min}, r_{\max}]}(r_k + \eta(e_{P,k} - e_{D,k})), \quad (1)$$

where $e_{P,k}$ and $e_{D,k}$ are nonnegative TTFT and TPOT violation pressures, respectively. Deadband, saturation detection,

and phase-length limits prevent a slow update from causing per-iteration oscillation. The reported results use a configured r ; they do not attribute their gains to the adaptive extension.

We evaluate this exchange with request-level joint-SLO attainment and goodput. For requests arriving in measurement window T , output length o_i , and thresholds S_T, S_P , we use

$$G = \frac{1}{T} \sum_i o_i \mathbf{1}[\text{TTFT}_i < S_T \wedge \text{TPOT}_i < S_P]. \quad (2)$$

Here TPOT is the per-request mean inter-token time, not a bound on every token interval. Goodput can differ at similar raw throughput when latency distributions fall on different sides of the joint threshold.

4. PD-TDM DESIGN

PD-TDM runs on one shared GPU group hosting one tensor-parallel model instance. Weights and KV blocks remain local. Before every model iteration, the scheduler selects phase P or D and only then invokes the corresponding batch planner. A P iteration advances unfinished prompts; a D iteration generates one token for each selected active sequence. This phase-purity invariant is enforced for running partial Prefills and newly admitted requests. The contribution is not phase separation alone, but making the phase service opportunity an explicit scheduling variable.

Bounded continuation. The prefill planner applies an aggregate $C = 2,048$ token limit per P iteration. It first continues partial prefills and then admits waiting requests with the remaining budget. An unfinished prompt retains its KV state and continues later. The token bound limits admitted work; since iteration time is workload dependent, it is not a strict wall-clock guarantee.

Phase-level allocator. A token bucket accumulates a configurable target Prefill tendency $r \in [0, 1]$. When both phases have work, sufficient credit selects P and consumes one unit; otherwise it selects D . If only one phase has work, the scheduler immediately serves it. Thus r describes a contention-time tendency, not a fixed sequence or wall-clock ratio. Minimum and maximum phase slice constraints debounce switching and bound starvation. A KV-pressure guard can override a planned P iteration, and optional TTFT-urgency or Decode-starvation signals can override the bucket when a request is near its SLO boundary.

Adaptive extension. The allocator has two distinct time scales. A fast selector makes one phase decision per iteration from the current queue snapshot and the bucket state. A slow loop can update r every W iterations from windowed TTFT/TPOT violation pressure, for example by applying Eq. 1 with EMA smoothing and saturation guards. This separates adaptation of the long-run phase tendency from immediate safety decisions. The T6 and MaaS measurements use the configured-target mode; the adaptive mode is not credited for the reported gains.

This design predicts its boundary. When prompt queueing dominates and TPOT has slack, concentrated P iterations can move TTFT left while D iterations remain within the TPOT threshold. Under a deep decode queue, the silence created by a P iteration consumes the remaining TPOT slack and mixed chunked prefill can be preferable.

5. EVALUATION

Setup. We use Qwen3-8B in BF16 on two Ascend 910B3 NPUs (64 GB each) with TP=2, CANN 8.3.rc1, PyTorch/torch-npu 2.7.1, and vLLM-Ascend v0.11.0rc1. The direct baseline, vLLM-Ascend CP, and PD-TDM use the same replica and 2,048-token prefill bound. T6 replays Azure-derived conversation and code bursts with a 10-s cycle. Mean prompt/output lengths are 1,155/211 and 2,048/28 tokens, respectively. Each point runs for 90 s with a 30 s warm-up; we report the median of three arrival seeds. Strict TTFT/TPOT thresholds are 200/120 ms for conversation and 500/200 ms for code. Metrics include requests whose arrivals fall in the measurement cohort. All reported runs use the final scheduler path with the admission/waiting fix applied.

Azure bursts: configured phase allocation. Fig. 1 shows the load sweep. At conversation scale $k = 3.0$, PD-TDM raises attainment from 29.0% to 63.8% (+34.8 points) and goodput from 1,525 to 3,396 output token/s. At code scale $k = 4.9$, attainment rises from 6.5% to 80.4% (+73.9 points) and goodput from 87 to 438 output token/s. The large relative goodput at this second point should be read with its low baseline attainment, not as a workload-independent speedup. Wider SLOs often place both systems near 100% attainment, indicating that the main effect is a latency-distribution shift rather than unconditional raw-throughput growth.

These measurements validate the phase-allocation operating point, not an online controller. In the reported runs, the target phase tendency is configured for the workload/SLO cell and then smoothed by the selector. They therefore show that explicit cross-iteration allocation can move requests across the joint SLO boundary, but they do not establish that a runtime controller can discover the best tendency without configuration.

Scheduler telemetry supports the execution claim without establishing a kernel cause. In one persistently backlogged code point, PD-TDM alternates pure P/D iterations and the decode-iteration inter-arrival P50/P99 is 189/203 ms. Mixed and pure iterations perform unequal work, so their durations do not establish a fixed mixed-kernel tax.

MaaS replay and boundary. We reconstruct request lengths from minute-level aggregates and replay a 76-minute arrival trace containing 35,441 submitted requests per system, with no client errors. The configured cohort window is [30, 34200) s; it contains 35,345 arrivals, of which 21 baseline and 3 PD-TDM records lack matched latency telemetry. Table 1 reports matched-record latency summaries. PD-TDM

Table 1. Supporting MaaS replay results; latency in ms.

Metric	vLLM-Ascend CP	PD-TDM
TTFT mean / P99	952 / 1,316	808 / 1,180
TPOT mean / P99	236 / 257	222 / 252
3× SLO attainment	99.997%	99.997%
1.2× attainment*	63.4%	92.6%
1.2× goodput*	99.9	150.5

*Post-hoc; goodput in output token/s.

lowers mean/P99 TTFT by 15.1%/10.4%. Under the original loose 3× ideal SLO, both systems are at the ceiling. Tightening the same traces to 1.2× ideal produces the listed separation, but this is a post-hoc sensitivity analysis rather than a prespecified service target.

Synthetic workloads expose the other side of the trade-off. Balanced loads show TTFT gains, whereas longer-output, decode-heavy cells become neutral and then favor the mixed baseline at high load. These boundary runs are single seed and some high-load cells have client timeouts, so we treat them as directional counterexamples rather than stable capacity estimates.

Adaptive-controller validation. [TODO: Compare mixed chunked prefill, the best fixed phase tendency, and the adaptive allocator in Eq. 1 under an in-trace workload or SLO change. Report the ratio trajectory, TTFT/TPOT response, and stability.] The required comparison is separate from the configured operating points above: an effective allocator should increase r when TTFT pressure rises, reduce r when Decode silence threatens TPOT, and avoid oscillation through the bucket and slice guards. No headline result in this section is attributed to the adaptive mode.

Limitations. The evidence is limited to Qwen3-8B, one two-NPU platform, one software version, and a fixed 2,048-token bound. The study does not establish the superiority of a best-tuned baseline over all possible chunk budgets, isolate mixed and phase-pure execution under equal work, or validate the adaptive controller across changing regimes. The MaaS input is reconstructed from aggregates and its tighter SLO is post hoc. Runtime and attention-backend changes can also alter the cost of mixed and phase-pure execution; our results therefore characterize the pinned integrated stack rather than a version-independent hardware effect. The evidence supports an end-to-end comparison of the evaluated deployments, but does not establish a universally optimal phase share or an inherent mixed-batch tax.

6. CONCLUSION

PD-TDM separates the bounded work inside a Prefill iteration from the cross-iteration allocation of service opportunities between Prefill and Decode. It keeps weights and KV state on one shared GPU group and makes phase selection explicit

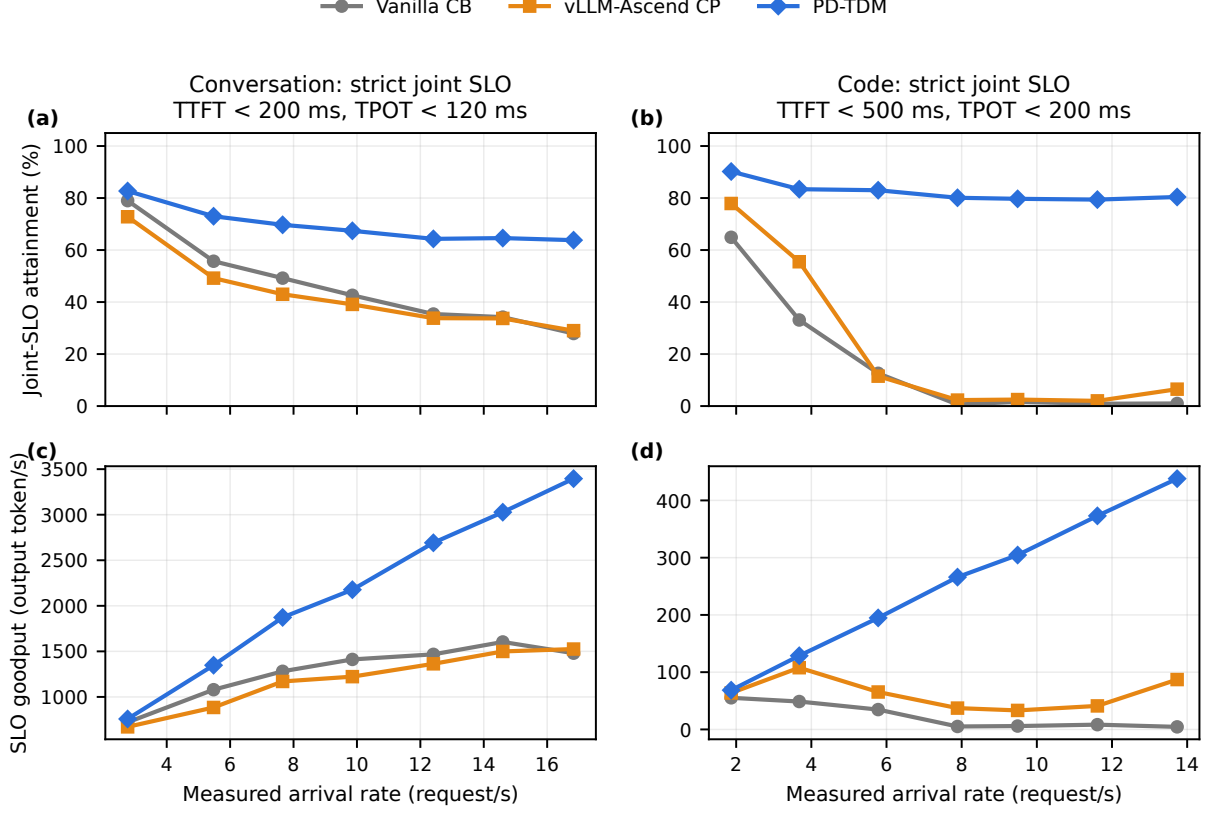


Fig. 1. Strict joint-SLO attainment and output-token goodput on Azure-derived bursts. Each point is the median of three seeds. vLLM-Ascend CP and PD-TDM use the same two-NPU TP replica and 2,048-token prefill bound.

before batch construction. On two Ascend 910B3 NPUs, the configured allocator improves joint-SLO attainment for Prefill-pressure bursts and shifts TTFT left on a reconstructed MaaS trace. Decode-heavy counterexamples provide the necessary qualification: allocating more Prefill service is useful when Decode has latency slack and can lose when TPOT is already tight.

The design admits a two-timescale extension that updates the target phase tendency from online SLO pressure. The reported evidence should be interpreted as validation of the explicit phase-allocation interface and its configured operating points, not as proof that the controller has found an optimal P/D ratio.

7. REFERENCES

- [1] Gyeong-In Yu, Joo Seong Jeong, Geon-Woo Kim, Soo-jeong Kim, and Byung-Gon Chun, “Orca: A distributed serving system for Transformer-Based generative models,” in *16th USENIX Symposium on Operating Systems Design and Implementation (OSDI)*, 2022, pp. 521–538.
- [2] Woosuk Kwon, Zhuohan Li, Siyuan Zhuang, Ying Sheng, Lianmin Zheng, Cody Hao Yu, Joseph E. Gonzalez, Hao Zhang, and Ion Stoica, “Efficient memory management for large language model serving with PagedAttention,” in *Proceedings of the 29th Symposium on Operating Systems Principles (SOSP)*, 2023, pp. 611–626.
- [3] Amey Agrawal, Nitin Kedia, Ashish Panwar, Jayashree Mohan, Nipun Kwatra, Bhargav Gulavani, Alexey Tumanov, and Ramachandran Ramjee, “Taming Throughput-Latency tradeoff in LLM inference with Sarathi-Serve,” in *18th USENIX Symposium on Operating Systems Design and Implementation (OSDI)*, 2024, pp. 117–134.
- [4] Connor Holmes, Masahiro Tanaka, Michael Wyatt, Ammar Ahmad Awan, Jeff Rasley, Samyam Rajbhandari, Reza Yazdani Aminabadi, Heyang Qin, Arash Bakhtiari, Lev Kurilenko, and Yuxiong He, “DeepSpeed-FastGen: High-throughput text generation for LLMs via MII and DeepSpeed-Inference,” *arXiv preprint arXiv:2401.08671*, 2024.
- [5] Pratyush Patel, Esha Choukse, Chaojie Zhang, Aashaka Shah, Íñigo Goiri, Saeed Maleki, and Ricardo Bianchini,

“Splitwise: Efficient generative LLM inference using phase splitting,” in *2024 ACM/IEEE 51st Annual International Symposium on Computer Architecture (ISCA)*, 2024, pp. 118–132.

- [6] Yinmin Zhong, Shengyu Liu, Junda Chen, Jianbo Hu, Yibo Zhu, Xuanzhe Liu, Xin Jin, and Hao Zhang, “Dist-serve: Disaggregating prefill and decoding for goodput-optimized large language model serving,” in *18th USENIX Symposium on Operating Systems Design and Implementation (OSDI)*, 2024, pp. 193–210.
- [7] Chao Wang, Pengfei Zuo, Zhangyu Chen, Yunkai Liang, Zhou Yu, and Ming-Chang Yang, “Prefill-decode aggregation or disaggregation? unifying both for goodput-optimized LLM serving,” *arXiv preprint arXiv:2508.01989*, 2025.
- [8] Ruoyu Qin, Zheming Li, Weiran He, Jiale Cui, Feng Ren, Mingxing Zhang, Yongwei Wu, Weimin Zheng, and Xinran Xu, “Mooncake: Trading more storage for less computation—a KVCache-centric architecture for serving LLM chatbot,” in *23rd USENIX Conference on File and Storage Technologies (FAST)*, 2025, pp. 155–170.
- [9] Yukang Chen, Weihao Cui, Han Zhao, Ziyi Xu, Xiaozhe Fan, Xusheng Chen, Yangjie Zhou, Shixuan Sun, Bingsheng He, and Quan Chen, “Towards high-goodput LLM serving with prefill-decode multiplexing,” *arXiv preprint arXiv:2504.14489*, 2026.
- [10] Ke Hong, Lufang Chen, Zhong Wang, Xiuhong Li, Qili Mao, Jianping Ma, Chao Xiong, Guanyu Wu, Buhe Han, Guohao Dai, Yun Liang, and Yu Wang, “semi-PD: Towards efficient LLM serving via phase-wise disaggregated computation and unified storage,” *arXiv preprint arXiv:2504.19867*, 2025.
- [11] Jiangsu Du, Hongbin Zhang, Taosheng Wei, Zhenyi Zheng, Jiazhi Jiang, Kaiyi Wu, Zhiguang Chen, and Yutong Lu, “Efficient LLM serving on commodity GPU clusters with Data-Reduced Cross-Instance orchestration,” in *20th USENIX Symposium on Operating Systems Design and Implementation (OSDI)*, 2026, pp. 1787–1802.